Demo video script: Situation Guard for Alexa+

Target length: 2:50 (hard limit 3:00). **Format:** screen recording with your voice-over, plus Alexa's spoken replies. Upload to YouTube as public or unlisted, in English.

The one idea judges must remember: *Alexa proposes, a strict engine decides, you approve. Alexa can't tell you a fix works unless it really does.*

Before you record (10 minutes, saves an hour)

1. **Run the dry run once** with the exact lines below. Fix anything odd before recording.
2. **Clean state.**
   - Delete yesterday's "My day" plans (Delete plan button).
   - Remove any test events from tomorrow in Google Calendar, so tomorrow at 2pm is free.
   - Click **New conversation**.
3. **`.env` is live:**
   - `SITUATION_GUARD_LLM=openai`
   - `TRAVEL`, `PLACES` and `WEATHER` set to `live`
   - `CALENDAR=google`
   - Restart the server.
4. **Browser:** Chrome, 1920×1080, zoom 100% (or 110% if text looks small), bookmarks bar hidden, notifications off.
   - Tab 1: the app.
   - Tab 2: Google Calendar, on tomorrow's **day** view.
5. **Sidebar check:** all four Connections dots are green (GPT-5 mini, Location, Weather & travel = Live, Calendar = Google Calendar).
6. **Speaker on**, so Alexa's voice is recorded. Use a headset microphone for your own voice-over if you can.
7. **Record each scene separately.** Alexa's replies take a few seconds, so cut the waiting in editing (keep the typing dots for half a second so it feels real).

> Alexa's exact words will differ from the examples below. That's fine. Keep what she really says: it's live.

Scene 1: Hook (0:00 – 0:12)

Screen: the app's welcome screen ("What's the plan?"), the Alexa ring glowing.

You say:

> "You know when you book something, then realise you've already got something on at that time? Happens to me all the time. So I built an Alexa that actually checks before she says yes"

On-screen text: Situation Guard for Alexa+

Scene 2: The idea in one picture (0:12 – 0:28)

Screen: a simple slide (or the README diagram): `You speak → Alexa (LLM) → MCP server → Feasibility engine → verdict → you approve`

You say:

> "Situation Guard is a self-hosted MCP server for Alexa+, with 17 tools. Alexa proposes, a deterministic engine decides, and nothing changes until you say yes. It's connected to the real world: my location, my Google Calendar, live weather, and real public transport from Amazon Location."

On-screen text: *Alexa proposes → Engine decides → You approve*

Scene 3: Real weather, warm-up (0:28 – 0:40)

Screen: the app. Click the mic and speak (it shows voice works).

Say to Alexa:

> "What's the weather today?"

Alexa answers with the live temperature where you are.

You say (over it, briefly):

> "Live weather, from my real location."

Scene 4: Book a meeting into my real calendar (0:40 – 1:00)

Type or say:

> "Schedule a meeting with my boss tomorrow at 2pm for an hour."

Screen: Alexa confirms. On the right, the timeline shows the meeting. Open **Activity** for one second: the "Added to your Google Calendar" step. **Switch to the Google Calendar tab:** the meeting is really there at 2pm.

You say:

> "The engine checked tomorrow for clashes, then added it to my actual Google Calendar."

Scene 5: The clash, caught and fixed (1:00 – 1:35), the heart of the demo

Back to the app. Type or say:

> "Tomorrow I also want to talk with one of my teammates at 2pm."

Alexa: tells you it clashes with the boss meeting and offers a tested alternative time, ending with a yes/no question.

Screen: the timeline shows the clash in **red**, "1 problem". Open **Activity**: the clash, the alternatives found, and the engine's verdict on each.

You say:

> "I can't be in two meetings at once. The engine catches it, Alexa finds alternatives, and every single one is tested before she offers it."

Say to Alexa:

> "Yes, do that."

Screen: the verdict turns **green: "Works"**. Activity shows **"verified"**.

You say:

> "Only after my yes does the plan change, and it's reloaded and re-checked to prove it."

Scene 6: Real trip, real transport (1:35 – 2:10)

Type or say:

> "Tomorrow at 2pm I want to go from here to Sydney Airport by public transport."

Alexa: finds real train routes, but they all overlap your 2pm meeting. She says so and offers a later one.

Screen: open **Activity**: - "Found 4 alternatives" (real T8 trains from Amazon Location) - red "doesn't work: you can't do two things at once" on each 2pm option

Zoom in on that list for 2 seconds.

You say:

> "These are real Sydney trains from Amazon Location Service. The engine rejects every one that clashes with my meeting. Alexa can't pretend otherwise."

Say to Alexa: "Yes." (accept the later trip)

Alexa: confirms, then checks the weather for when you leave, e.g. "*Dry when you leave at 3pm*" or "*Take an umbrella.*"

You say:

> "And she checks the weather for the moment I walk out the door."

Scene 7: Under the hood (2:10 – 2:35)

Screen, in quick cuts: 1. The **Activity** panel scrolled through: every MCP tool call with its verdict. 2. The README's tools list or the MCP endpoint `localhost:8000/mcp`. 3. The terminal: `pytest -q`, then **213 passed**.

You say:

> "Every step is a real MCP tool call over Streamable HTTP, the same server Alexa+ or any MCP client can use. The engine checks deadlines, shared resources, one person in two places, past times, closed places and rain. Consent is enforced in code, not just in the prompt. And it's covered by 213 offline tests."

Scene 8: Impact and close (2:35 – 2:50)

Screen: back to the timeline showing "Works", then the welcome screen with the scenario cards (TV, meetings, rain, public holiday).

You say:

> "Meetings, trips, a shared TV, rain, public holidays: one engine for all of them. Situation Guard turns Alexa from an assistant that sounds right into one you can trust with your day."

On-screen text: Situation Guard for Alexa+, the GitHub link, *Alexa proposes. The engine decides. You approve.*

Why this script scores

Judging criterion	Where it shows
Stage 1: theme and tech	"Self-hosted MCP server for Alexa+" in the first 20 seconds; MCP tool calls on screen in Scenes 4–7
Technical implementation	Real Google Calendar write, real Amazon Location transit, engine rejecting options, verified confirm, 213 tests
Design	Clean three-pane app, red to green timeline, Activity panel, voice in and out
Potential impact	Everyday problems everyone has: double bookings, late trains, rain. Generic, not one niche
Quality of idea	The hook: an Alexa that *can't* lie about your plans, propose / decide / approve
"Creative" per the rules	Multi-step agentic workflow across services, state kept across sessions, consent before action

If something goes wrong while recording

- **Alexa asks an unexpected question:** answer it naturally, then cut it if it's slow.
- **A reply is wrong or takes too long:** click **New conversation**, delete the plan, and re-record just that scene.
- **No train routes found:** check the sidebar says Live, and that "Sydney Airport" was found (the place-search permission).
- **Over 3:00:** shorten Scene 3 (weather) first, then trim the pauses in Scene 7.